



CHAPTER-2-PROBLEM SOLVING

Wattanapong Suttapak



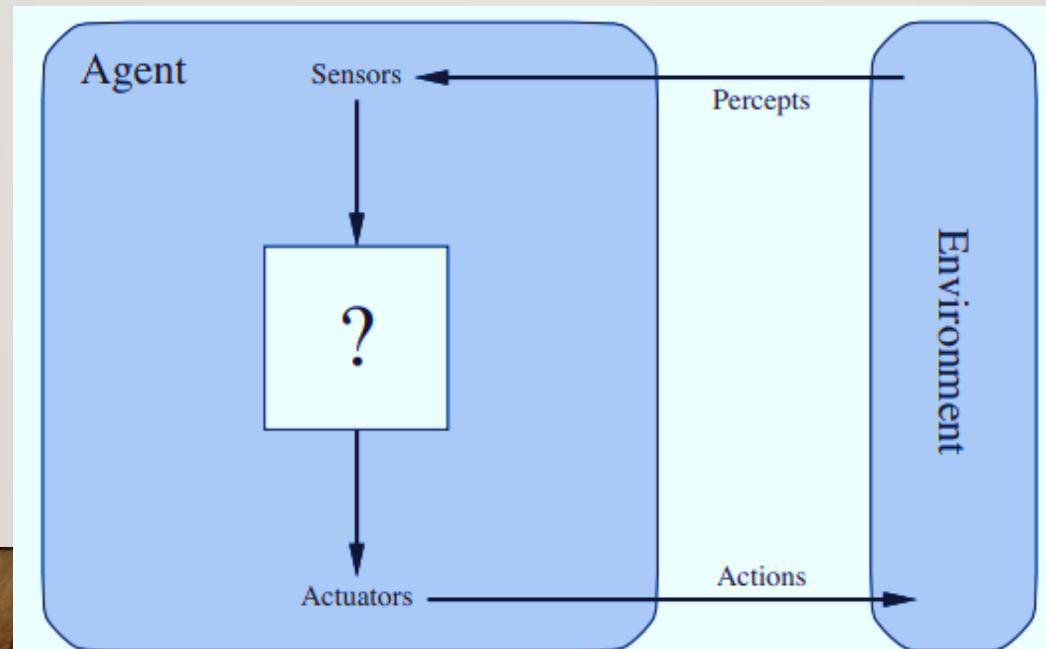
2 OUTLINE

- Definition of Rational Agent
 - Agent, Performance Measure
- Deterministic and Stochastic Problems
- Performance Measurement
- Problem Solving
 - Uninformed Search
 - Breadth and Depth First Search
 - Informed Search
 - Greedy and A* Search
 - Local Search
 - Optimization
 - Adversarial Search
 - Non-deterministic Problem Solving

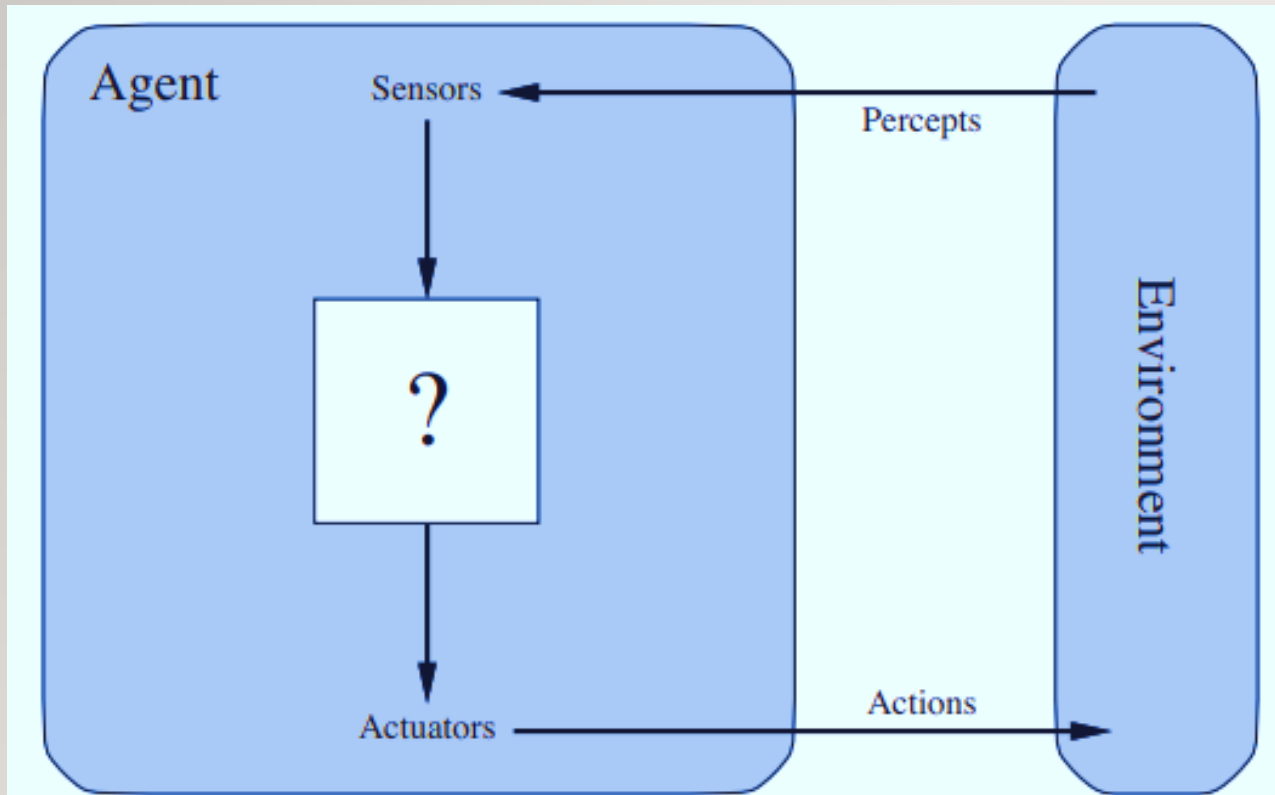
3 AGENT

AGENT

ตัวแทน (**Agent**) คือสิ่งใดก็ตามที่สามารถรับรู้สภาพแวดล้อม (**Environment**) ผ่านตัวรับสัญญาณ (**Sensors**) และตอบสนองผ่านตัวกระทำ (**Actuators**) ไปสู่สภาพแวดล้อม ดังรูป



4 AGENT



Environment: sending or receiving to agent

Percepts: sending information to agent

Sensors: the component that receives information from a perception through agent

Getting

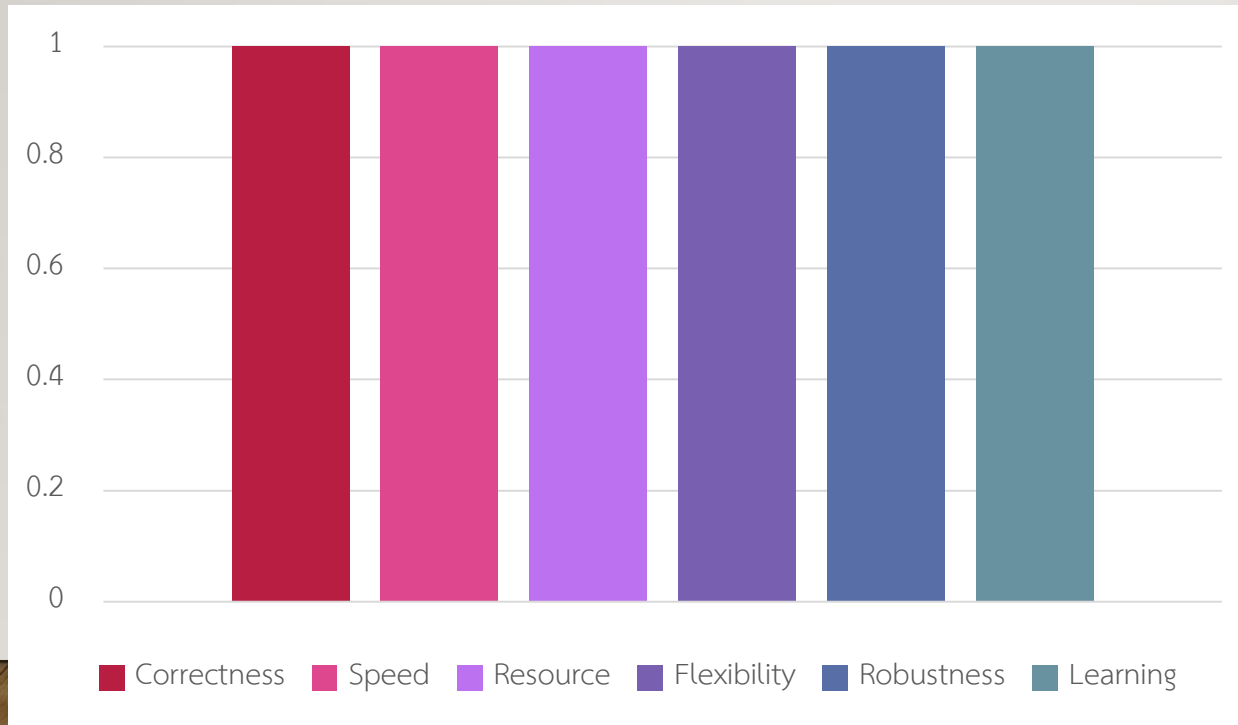
Actuators: the component that transfers information as acting

Actions: the responding into environment

Responding

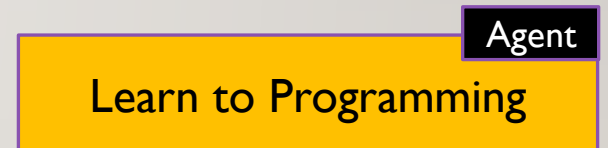
5 PERFORMANCE MEASUREMENT

Performance Measurement เป็นกระบวนการในการประเมินว่า Agent ทำงานได้ดีเพียงใดภายใต้เป้าหมายหรือภารกิจที่กำหนดไว้ ซึ่งถือเป็นหัวใจสำคัญของการออกแบบและพัฒนา Agent ให้มีความฉลาดจริง



1. ความถูกต้องของผลลัพธ์ (Correctness / Goal Achievement)
2. ความเร็วในการตัดสินใจ (Efficiency / Speed)
3. การใช้ทรัพยากร (Resource Usage)
4. ความยืดหยุ่น (Flexibility / Adaptability)
5. ความทนทาน (Robustness)
6. ความสามารถในการเรียนรู้ (Learning Ability)

6 PERFORMANCE MEASUREMENT



Environment	Perception	Action	Performance
C language	$i = 0$	$i = 0;$	Compile Success
C language	$i = 0$	$i = 2$	Compile Error

7 RATIONALITY

- ตัวแทนที่มีเหตุผล (Rational Agent) คือ ตัวแทนที่เลือกการกระทำที่ เพิ่มค่าคาดหวังของผลลัพธ์ ให้ได้ผลลัพธ์ที่ดีที่สุด (Maximize Expected Performance) จากข้อมูลที่ได้รับ ณ เวลานั้น
- เราคาดหวังว่าตัวแทนที่มีเหตุผลจะสามารถให้ผลลัพธ์ในระดับผู้รอบรู้ (Omniscience) กล่าวคือ ความคาดหวังให้รู้อนาคตล่วงหน้าและทำผลลัพธ์ที่ดีที่สุดเสมอ (Actual Performance) ซึ่งเป็นไปไม่ได้ในโลกจริง เนื่องจากสิ่งแวดล้อมที่ส่งผลต่อตัวแทน และจำนวนความน่าจะเป็นทั้งหมดมีจำนวนมหาศาล และ มีความซับซ้อนที่สุด ดังนั้น ตัวแทนที่มีเหตุผลมีเป้าหมาย คือ การใช้ข้อมูลที่ได้รับได้ในขณะนั้น ตัดสินใจโดยหวังผลลัพธ์ที่ดีที่สุด (Expected Performance)

8 RATIONAL AGENT VS OMNISCIENCE

แนวคิด	Rationality (มีเหตุผล)	Omniscience (รู้ทุกอย่าง)
พื้นฐานการตัดสินใจ	จากสิ่งที่รับรู้ได้ (Perceptions)	จากข้อมูลสมบูรณ์ทุกด้าน
ประสิทธิภาพ	สูงสุดที่คาดว่าจะเป็นไปได้	สูงสุดทุกความเป็นไปได้
ระดับการใช้งาน	เฉพาะข้อมูลที่ได้รับรู้ได้	ทุกสถานการณ์

9 RATIONALITY

- Rationality $\neq$ Omniscience
- Rationality ทำงานบนพื้นฐานของการรับรู้ข้อมูล (perceptions) และ ข้อเท็จจริง (facts)
- ตัวอย่างเช่น

การเดินข้ามถนน — เรามองซ้ายและขวาก่อนข้าม นี่คือตัวแทนที่มีเหตุผล แต่เราไม่สามารถรู้ได้ว่าอุบัติเหตุจะเกิดขึ้นหรือไม่ ซึ่งแสดงให้เห็นว่าเราตัดสินใจอย่างมีเหตุผลจากข้อมูลที่เราได้อยู่ ณ ขณะนั้น

10 AGENT EXAMPLES

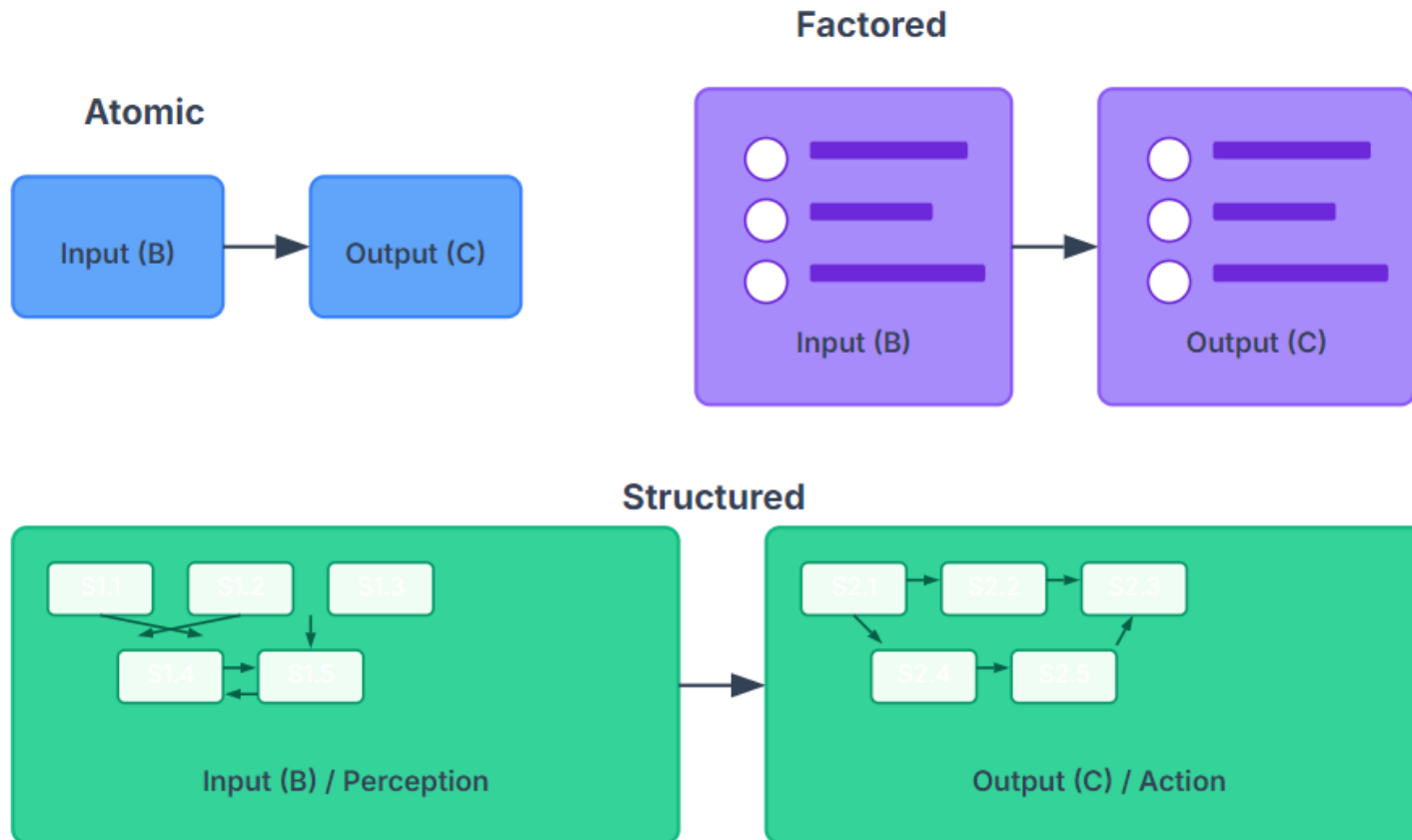
หมวดหมู่ (Agent Type)	Taxi Driver Agent (เอเจนต์คนขับแท็กซี่)	Medical Diagnosis System (ระบบวินิจฉัยโรค)
สภาพแวดล้อม (Environment)	ถนน, การจราจร, สภาพอากาศ, ผู้โดยสาร, กฎหมายจราจร	ห้องตรวจ, ข้อมูลผู้ป่วย, ประวัติการรักษา, โรคและอาการต่าง ๆ
ตัวรับข้อมูล (Sensors)	กล้อง, GPS, ตัววัดความเร็ว, แผนที่, การสนทนากับผู้โดยสาร	คีย์บอร์ด, แบบสอบถามสุขภาพ, เครื่องวัดสัญญาณชีพ, ผลตรวจแล็บ, X-ray
ตัวกระทำ (Actuators)	พวงมาลัย, เบรก, คันเร่ง, สัญญาณไฟ, ระบบนำทาง, การโต้ตอบกับผู้โดยสาร	หน้าจอแสดงผล, รายงานผลวินิจฉัย, คำแนะนำการรักษา, การส่งต่อแพทย์
เกณฑ์การวัดประสิทธิภาพ (Performance Measurement)	- ไปถึงจุดหมายอย่างปลอดภัย - ใช้เส้นทางที่มีประสิทธิภาพ - ประหยัดเวลาและเชื้อเพลิง	- ตอบสนองต่อเหตุการณ์ฉุกเฉินได้ดี - วินิจฉัยโรคได้ถูกต้องแม่นยำลดความผิดพลาดในการรักษา - ประมวลผลได้รวดเร็วปรับตัวตามผู้ป่วยหรือข้อมูลใหม่ได้ดี

II AGENT EXAMPLES

หมวดหมู่ (Agent Type)	สภาพแวดล้อม (Environment)	ตัวรับข้อมูล (Sensors)	ตัวกระทำ (Actuators)	เกณฑ์การวัด ประสิทธิภาพ
Programmer	?	?	?	?

12 COMPONENTS OF AGENT PROGRAMS

Components of Agent Programs



13 COMPONENTS OF AGENT PROGRAMS

- **Atomic**

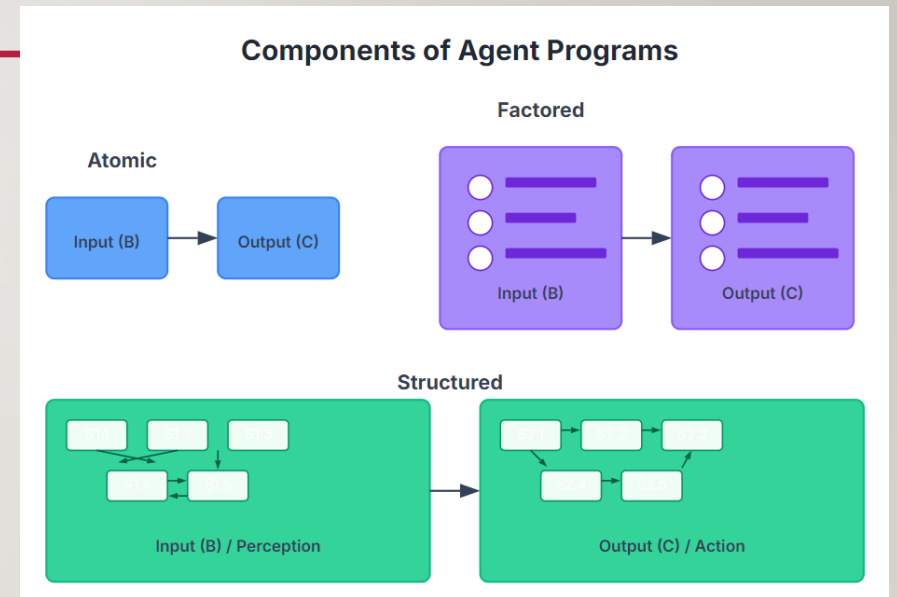
We only know the input and desire output (Blackbox)

- **Factored**

We further know elements of input and output (Blackbox)

- **Structured**

We know the relation between related elements of input and output (Whitebox)



PROBLEM SOLVING BY SEARCHING



PROBLEM SOLVING

- ปัญหาในงานด้านปัญญาประดิษฐ์ มักเริ่มต้นจากการพยายามแก้โจทย์ปัญหาให้ได้
- หลังจากการแก้ปัญหาได้ จะพยายามคิดวิธีแก้ปัญหที่พัฒนาขึ้นจากวิธีเดิม จนกว่าคำตอบที่ได้จะเป็นคำตอบที่ดีที่สุด (ต้องพิสูจน์ด้วยคณิตศาสตร์ได้)
- โดยปกติปัญหา (ด้านปัญญาประดิษฐ์) ไม่มีโครงสร้างที่สามารถแก้ไขได้ชัดเจน
- มีการคิดค้นเทคนิคการแปลงปัญหาให้อยู่ในรูปแบบโครงสร้างที่ชัดเจนมากขึ้น เรียกว่า ตัวแทนปัญหา (Problem Representation)
- ส่วนมากจะนิยมใช้ การแทนปัญหาด้วยปริภูมิ (มิติ) สถานะ (State Space Representation)
- การค้นหาคำตอบ (Search) ถูกนำมาใช้แก้โจทย์ปัญหา
- กรณีที่พบคำตอบมากกว่าหนึ่งวิธี ขั้นตอนต่อไปคือการค้นหาเส้นทางเพื่อไปยังคำตอบที่ดีที่สุด

PROBLEM SOLVING STEPS

- หลักการในการพิจารณาขั้นตอนการแก้ไขปัญหามีทั้งหมด 6 ขั้นตอน
 - Define the Problem ทำความเข้าใจปัญหา หานิยามขององค์ประกอบของปัญหา
 - Choose a search strategy เลือก Algorithm ที่เหมาะสมในการแก้ปัญห
 - Implement search algorithm เขียนโค้ด
 - Handle edge case and optimization จัดการความน่าจะเป็นที่คาดไม่ถึง (infinite loop, buffer overflow) และปรับปรุงโค้ด
 - Verify and Validate ทดสอบ (จากข้อมูลเดิม) และทวนสอบ (ด้วยข้อมูลทดสอบใหม่)
 - *Iterate and Improve ทำซ้ำ และปรับปรุงโค้ดให้ดีขึ้น (refinement) **

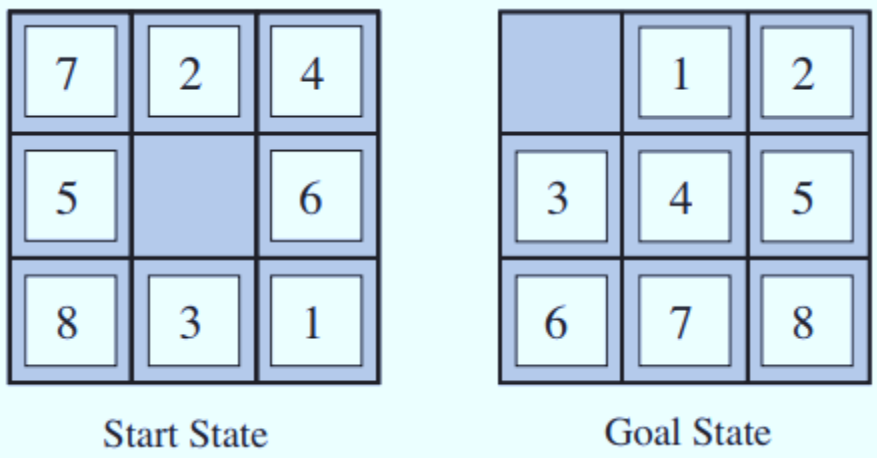
* มักใช้กับปัญหาที่ซับซ้อน เช่น ปัญหาที่ต้องใช้วิธีการหาค่าที่เหมาะสม (Optimization) หรือปัญหาด้านการเรียนรู้ของเครื่องจักร (Machine Learning)

STATE SPACE REPRESENTATION

- การนำปัญหามาแปลงให้อยู่ในรูปมาตรฐาน (Well-defined Problem) ประกอบด้วย
 - **Initial State** กำหนดองค์ประกอบ สิ่งแวดล้อมของปัญหา และกำหนดสถานะเริ่มต้นของปัญหา
 - **Action** กำหนดเซตของการกระทำทั้งหมดที่เป็นไปได้
 - **Goal State** กำหนดสถานะเป้าหมายของปัญหา
 - **Transition Model** การเปลี่ยนสถานะ
 - **Path Cost** กำหนดค่าใช้จ่ายหรือทรัพยากรที่ใช้ทั้งหมด จากการกระทำแต่ละครั้ง

18 ตัวอย่างปัญหา

Toy Problem: 8 puzzles



State

8-puzzle ประกอบด้วยแผ่นกระดาน 3×3 แต่ละแผ่นป้ายมีหมายเลขกำกับที่ไม่ซ้ำกันตั้งแต่ 1 ถึง 8

Initial State

สถานะเริ่มต้น Start State

Action

ช่องว่างสามารถขยับได้ 4 ทิศทาง {Up, Down, Left, Right}

Goal State

สถานะเป้าหมาย Goal State

Path Cost

ทุก ๆ การขยับช่องว่าง นับ 1 การเคลื่อนที่

19 ตัวอย่างปัญหา

Toy Problem: n-puzzle

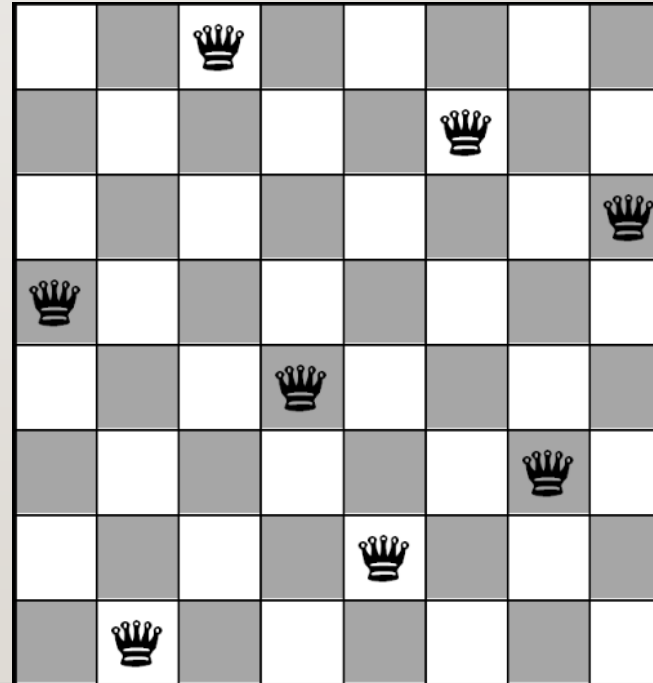
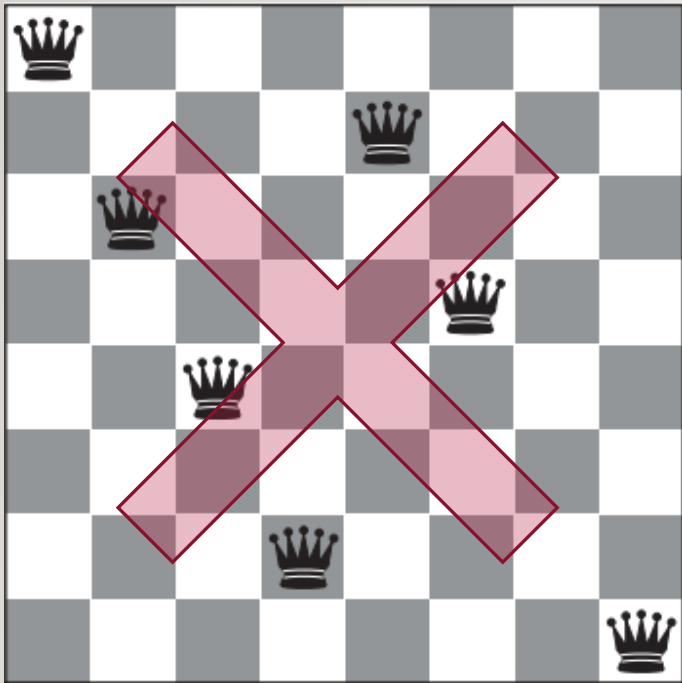
3x3 puzzles = $9! / 2 = 181,440$ states

4x4 puzzles = 1.3 trillion states

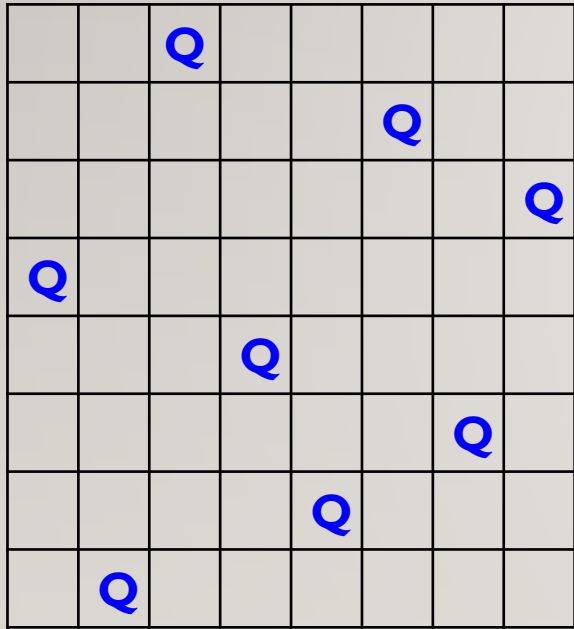
5x5 puzzles = 10^{25} states

20 ตัวอย่างปัญหา

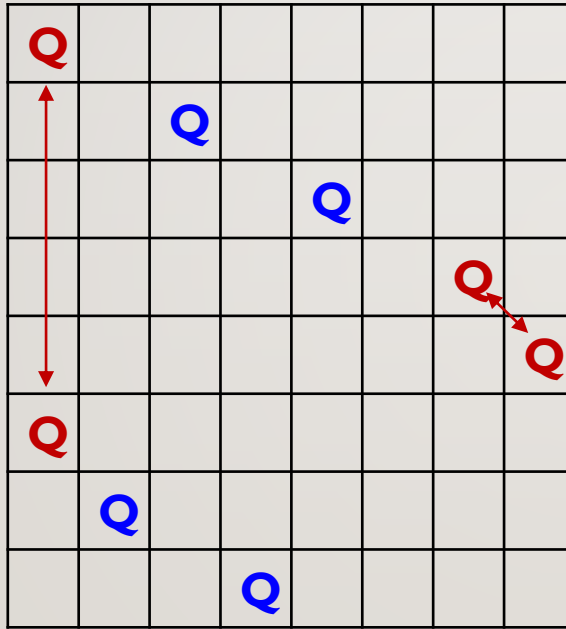
- **Toy Problem: 8 queens**



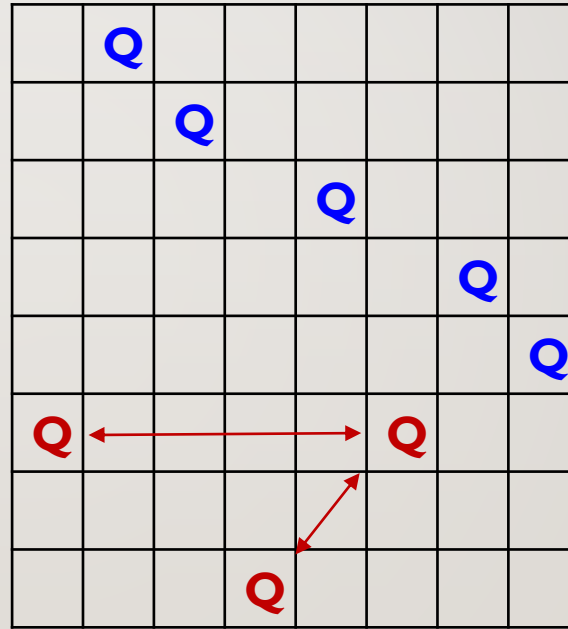
21 8-QUEENS ILLEGAL-MOVES EXAMPLE



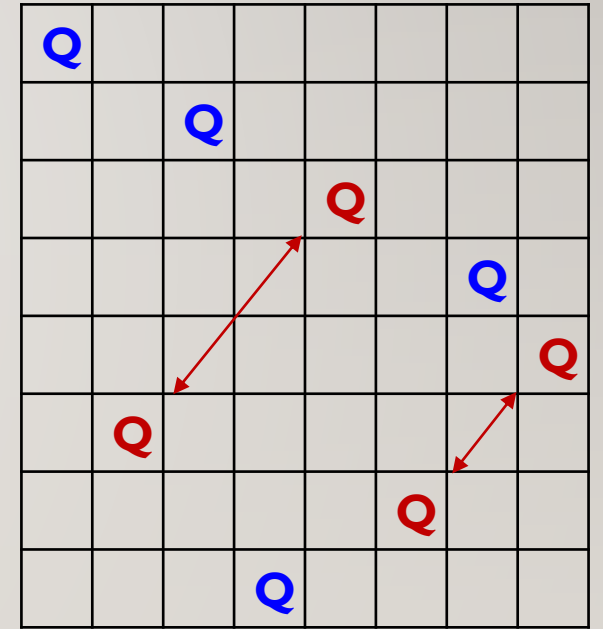
Legal



Illegal



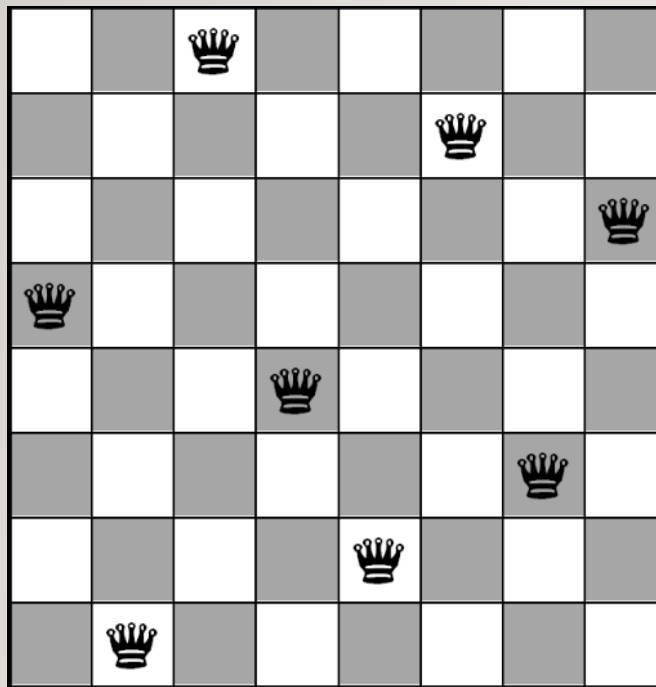
Illegal



Illegal

22 ตัวอย่างปัญหา

- **Toy Problem: 8 queens**



State

8-queen ประกอบด้วยแผ่นกระดาน 8×8 แต่ละแผ่นป้าย
สามารถวาง หมาก queen ได้หนึ่ง

Initial State

สถานะเริ่มต้น วาง queen ได้ 1 ตำแหน่ง จากทั้งหมด 64
ตำแหน่ง

Action

วาง queen ตัวถัดไป บนกระดาน ที่ไม่ซ้ำตำแหน่งเดิม

Goal State

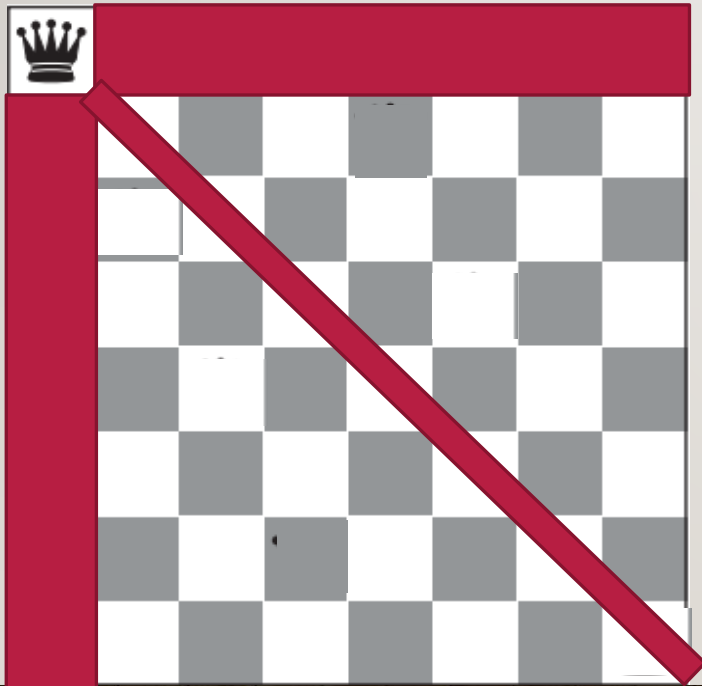
สถานะเป้าหมาย วาง queen ได้ 8 ตัว โดยทุกตัว จะต้องไม่อยู่บน
แถว คอลัมน์ หรือแนวทแยงมุม

Path Cost

ไม่นับ cost

23 ตัวอย่างปัญหา

- **Toy Problem: 8 queens**



วิธี Bruteforce

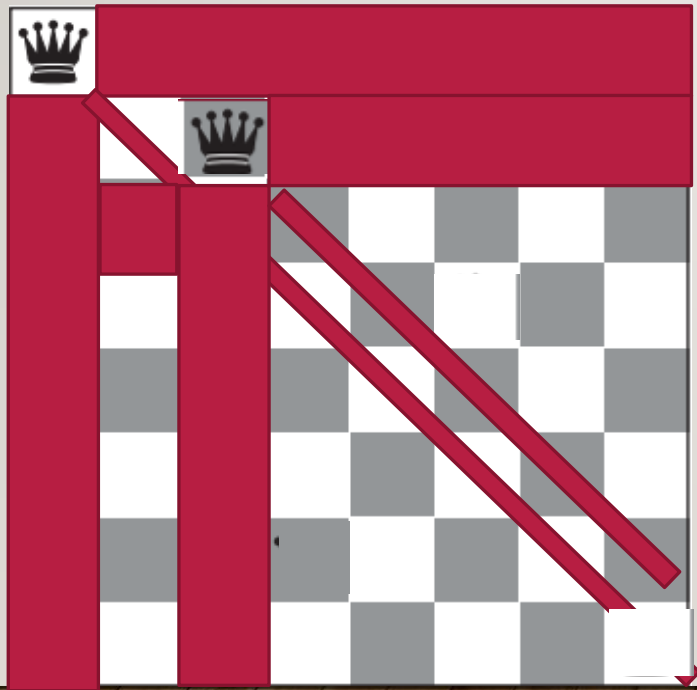
- ในการหาคำตอบทั้งหมดของ 8-queen จะใช้ความน่าจะเป็นในการخاب ทั้งหมด $64 \times 63 \times 62 \times \dots \times 1 = 1.8 \times 10^{14}$ ครั้ง

วิธี ตัดตำแหน่งที่ทับซ้อนออก

- ความน่าจะเป็นในการخاب เท่ากับ **2057** ครั้ง

24 ตัวอย่างปัญหา

- **Toy Problem: 8 queens**



วิธี Bruteforce

- ในการหาคำตอบทั้งหมดของ 8-queen จะใช้ความน่าจะเป็นในการขยับ ทั้งหมด $64 \times 63 \times 62 \times \dots \times 1 = 1.8 \times 10^{14}$ ครั้ง

วิธี ตัดตำแหน่งที่ทับซ้อนออก

- ความน่าจะเป็นในการขยับ เท่ากับ **2057** ครั้ง

$64 \times 6 \dots$

DETERMINISTIC AND NON-DETERMINISTIC PROBLEMS



26 DETERMINISTIC AND NON-DETERMINISTIC PROBLEMS

ในทาง **AI** สามารถแบ่งกลุ่มปัญหาออกเป็น 2 รูปแบบดังนี้

- **Deterministic problem:**

- ผลลัพธ์จากการป้อนอินพุต (**a**) ณ สถานการณ์ (**s**) ในปัญหาแบบ **Deterministic** จะนำไปสู่สถานการณ์เดิมทุกครั้ง (**s'**)
- $\text{Result}(s, a) = s'$
- Time complexity เท่ากับ polynomial time ($O(n^k)$)

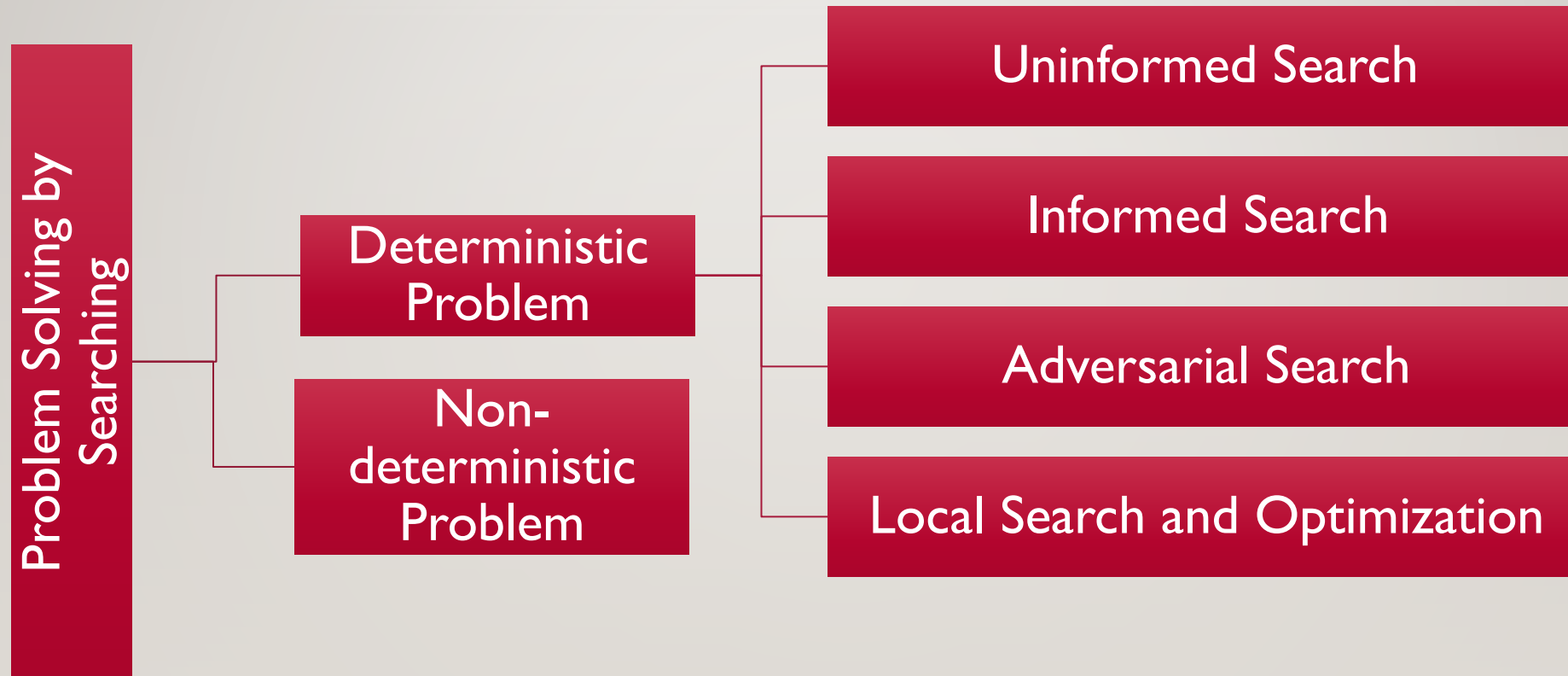
- **Otherwise: non-deterministic problem (stochastic problem)**

- ผลลัพธ์จากอินพุตที่ใส่ลงไปในฟังก์ชัน แต่ครั้ง ไม่เหมือนเดิม
- When $f(x)^{t=1} \neq f(x)^{t=2} \neq f(x)^{t=n}$

27 PROBLEM SOLVING BY SEARCHING

- **Deterministic Problem** สามารถแบ่งรูปแบบการค้นหาคำตอบ ออกเป็น 3 ประเภทใหญ่ ๆ
 - **Blind Search (Uninformed Search)** ไม่มีข้อมูลเพิ่มเติมในการค้นหา ทุกการค้นหามีโอกาสเท่ากันหมด
 - **Heuristic Search (Informed Search)** มีข้อมูลเพิ่มเติมในการค้นหา โอกาสค้นหาเป็นไปตามเงื่อนไขของข้อมูลที่มี
 - **Adversarial Search** การค้นหาวิธีกับปัญหาที่มีคู่ต่อสู้ (มักพบในเกมส์) **Opponent, Adversary**
- โดยมีการวัดประสิทธิภาพของการค้นหา (**Performance Measurement**) คือ
 - **Completeness** สามารถรับรองการค้นพบคำตอบ
 - **Optimality** สามารถรับรองการค้นหาเส้นทางที่ดีที่สุด
 - **Time Complexity** ระยะเวลาที่ใช้ในการค้นหา
 - **Space Complexity** พื้นที่หน่วยความจำที่ใช้ค้นหา

28 SEARCHING DIAGRAM



29 TIME COMPLEXITY

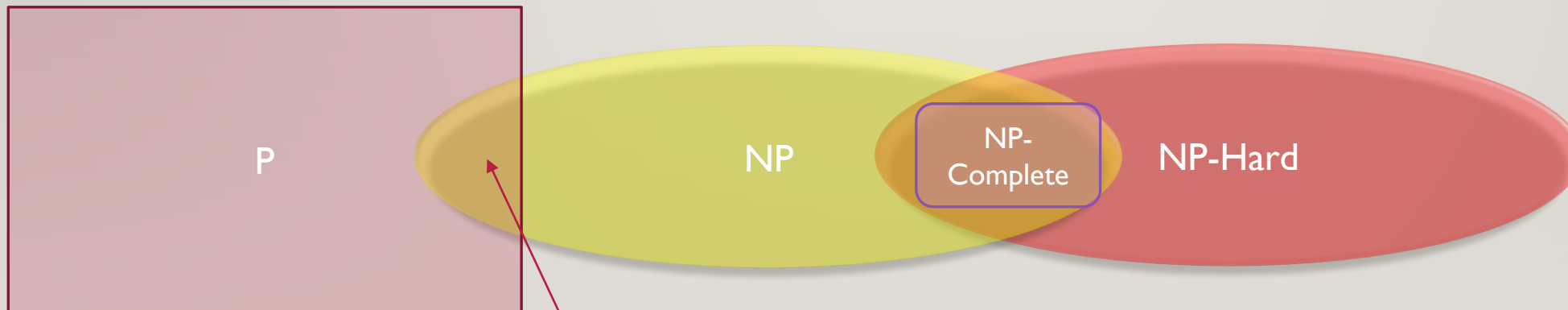
- Constant time $O(k)$
- Logarithmic time $O(\log_k(n))$
- Linear time $O(kn)$
- Quadratic time $O(n^2)$
- Polynomial time $O(n^k)$
- Exponential time $O(k^n)$
- Factorial time $O(n!)$

k = ค่าคงที่ (constant)

n = จำนวนของข้อมูล

30 COMPLEXITY THEORY

- P เป็นตัวแทนของ Problems ที่สามารถหาคำตอบได้ง่าย
- NP (nondeterministic polynomial time) เป็น Problem ที่สามารถหาคำตอบได้ยาก แต่ก็ยังสามารถหาคำตอบได้ แม้คำตอบที่ได้จะอยู่ใน polynomial time ก็ตาม
- NP-complete เป็นประเภท Problem ของ NP ที่ยากที่สุด



เริ่มต้น NP แต่สามารถแก้ปัญหาด้วย P ได้

3 | COMPLEXITY THEORY

คำศัพท์	ความหมาย	ตัวอย่างปัญหา
P (Polynomial Time)	ปัญหาที่ "แก้ได้ง่าย" และ "ตรวจสอบคำตอบได้ง่าย" ใช้เวลาแก้ปัญหาในระดับพหุนาม เช่น n^2 , n^3 เป็นต้น	- การเรียงลำดับข้อมูล (Sorting) - หาค่าเฉลี่ย - หาเส้นทางสั้นที่สุด (Dijkstra)
NP (Nondeterministic Polynomial Time)	ปัญหาที่ "อาจจะแก้ยาก" แต่ "ถ้าให้คำตอบมาแล้ว ตรวจสอบได้ง่าย" ไม่มีวิธีที่ตายตัวที่สามารถแก้ปัญหาได้อย่างรวดเร็วเสมอ แต่ถ้าวิธีใดที่สามารถหาคำตอบได้จะสามารถตรวจสอบได้ในเวลาไม่เกิน n^k	- ซูโดกุ (Sudoku) - ผลรวมเป้าหมาย (Subset Sum)
NP-Complete	กลุ่มของปัญหาที่อยู่ใน NP และ กลุ่มปัญหายากที่สุดใน NP ถ้าแก้ปัญหา NP-Complete ได้หนึ่งปัญหาเท่ากับแก้ได้ทุกปัญหาใน NP (นำวิธีการนั้นมาใช้กับปัญหาอื่น ๆ ใน NP)	- Traveling Salesman Problem (TSP) - Knapsack Problem
NP-Hard	ปัญหาที่ยากเท่ากับหรือยากกว่า NP-Complete ไม่จำเป็นต้องอยู่ใน NP (อาจตรวจสอบไม่ได้เร็ว) แต่ถ้าแก้ปัญหาได้ถือว่าประสบความสำเร็จ	- หมากรุก - โกะ - TSP (ค่าที่ดีที่สุด)

32 SORTING

- ตัวอย่างปัญหาการเรียงลำดับตัวเลข ที่มีการพัฒนาอัลกอริทึมในการค้นหา จาก $O(n^2)$ ไปสู่ $O(n \log n)$
- BUBBLE sort $O(n^2)$ -> Polynomial time $O(n^k)$
 - NP (Nondeterministic polynomial time)
- Merge sort $O(n \log n)$
 - P (Polynomial time)